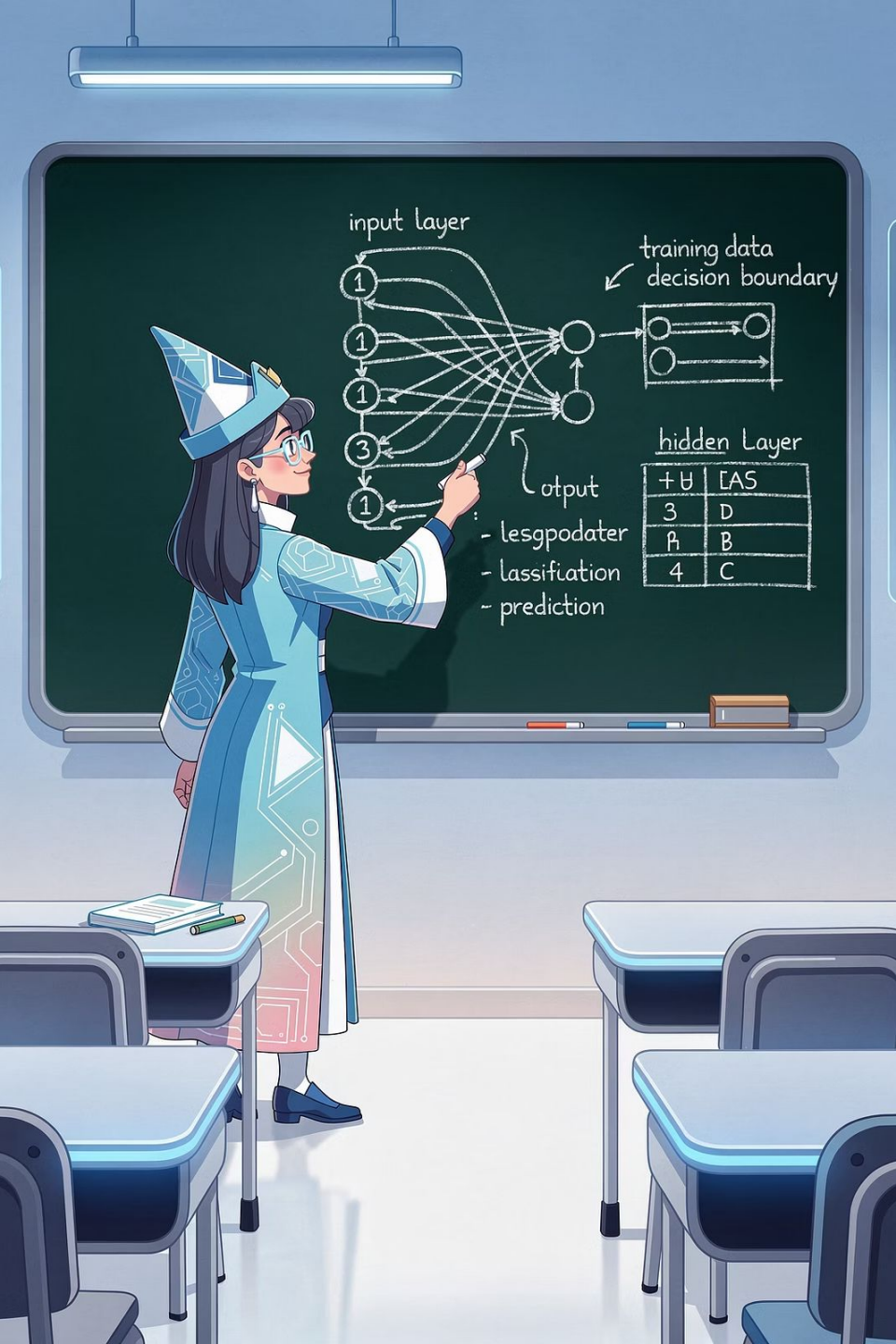




Neural Networks in R: How the Multilayer Perceptron Works

Classification with Neural Networks

Artificial neural networks are inspired by biological neural systems, offering a powerful approach to complex classification problems. These computational models can learn and model intricate non-linear relationships between variables, making them ideal for tasks where traditional methods fall short.

Biological Inspiration

Model the behavior of real neurons through interconnected computational units

Non-Linear Relationships

Capture complex patterns that are impossible for linear classifiers

Learning via Backpropagation

An iterative algorithm that adjusts weights to minimize prediction errors

Foundation of Deep Learning

Modern architectures evolved from these mathematical foundations

Originally developed by psychologists and neurobiologists seeking to create computational analogs of neurons, neural networks consist of sets of interconnected input/output units. Each connection has an associated weight that is adjusted during the learning phase, enabling the network to correctly predict the class label of input tuples. This approach is also known as connectionist learning due to the connections between units.

The Neuron: Hidden/Output Layer Unit

Mathematical Structure

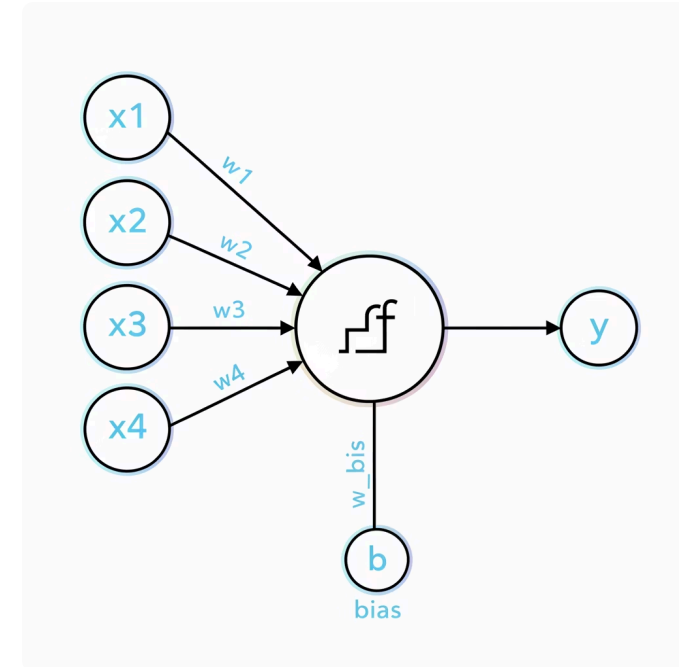
An artificial neuron maps an n-dimensional input vector $\mathbf{x}$ into an output variable $\mathbf{y}$ through two fundamental operations:

1. **Weighted Dot Product:** Inputs are multiplied by their corresponding weights and summed
2. **Non-Linear Activation Function:** The result is transformed by a function that introduces non-linearity

The inputs to each unit are the outputs of the previous layer. These inputs are multiplied by their corresponding weights to form a weighted sum, which is then added to the **bias** associated with the unit. Finally, a non-linear activation function is applied to this accumulated value.

Importance of Bias

The bias term allows the neuron to shift the activation function, increasing the model's flexibility to fit the training data.



Common Activation Functions:

- Sigmoid: $\sigma(x) = 1/(1+e^{-x})$
- ReLU: $f(x) = \max(0, x)$
- Tanh: $\tanh(x)$

How a Multilayer Neural Network Works

A multilayer neural network processes information through a layered architecture, gradually transforming input features into final predictions. Let's explore this process in detail:

Input Layer

Inputs correspond to the attributes measured for each training tuple and are fed in simultaneously

Hidden Layers

Data is weighted and processed through one or more hidden layers that extract meaningful features

Output Layer

The weighted outputs of the last hidden layer produce the network's final prediction

Architectural Characteristics

- **Feed-Forward:** The network is fed forward — no weight loops back to an input unit or to an output unit of a previous layer
- **Number of Hidden Layers:** Although arbitrary, typically only one hidden layer is used for moderately complex problems
- **Approximation Capacity:** From a statistical standpoint, neural networks perform non-linear regression

❏ **Universal Approximation Theorem:** With enough hidden units and adequate training samples, neural networks can approximate any continuous function to arbitrary precision.

Multilayer Feed-Forward Neural Network

Input Vector (X)

Represents the attributes of the input data, with each dimension feeding one neuron in the input layer

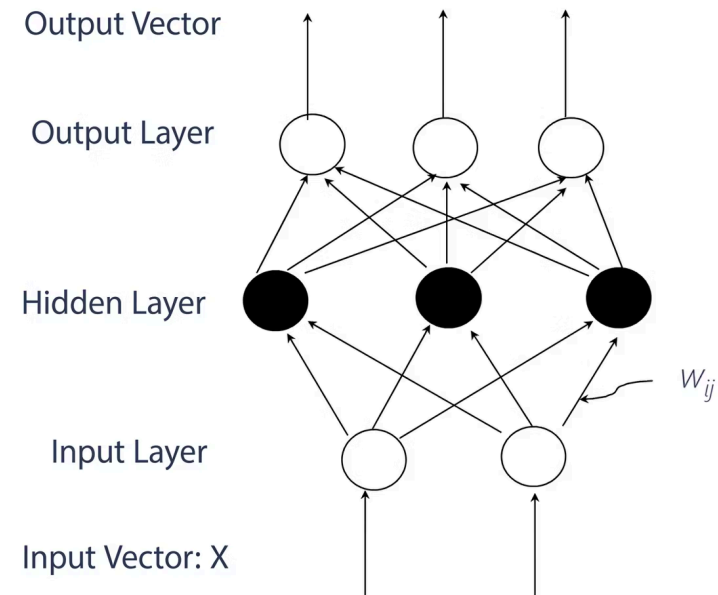
Weights (w_{ij})

Adjustable parameters that determine the strength of connections between neurons in adjacent layers

Output Vector

Produces the network's final predictions, with its shape depending on the problem type (classification or regression)

This multilayer architecture illustrates the flow of information through the network. The **input layer** receives raw data, the **hidden layers** extract increasingly abstract representations, and the **output layer** generates the final prediction. The weights w_{ij} between layers are optimized during training through the backpropagation algorithm.



Defining the Network Topology

Configuring the neural network topology is a critical step that directly influences model performance. Poor architectural decisions can lead to underfitting or overfitting the data.

Specify the Architecture

Define the number of units in the input layer, the number of hidden layers, units per hidden layer, and units in the output layer

Normalize Input Values

Normalize input values for each attribute measured in the training tuples to the range [0.0–1.0] to stabilize training

Configure Input Units

Create one input unit per domain value, each initialized to 0 to begin the learning process

Define the Output Layer

For classification with more than two classes, use one output unit per class; for regression, adjust accordingly

Iterate and Refine

If accuracy is unacceptable after training, repeat with a different topology or a different set of initial weights

 **Practical Tip:** Start with a simple architecture and gradually increase complexity. Cross-validation helps find the optimal topology without overfitting.

MLP Neural Network – Theory

The Multilayer Perceptron (MLP) is a foundational type of feedforward neural network specifically designed for classification tasks. It extends the basic perceptron by incorporating one or more hidden layers, enabling it to model complex, non-linear decision boundaries crucial for many real-world problems.

Core Definition

A feedforward Artificial Neural Network (ANN) characterized by one or more hidden layers between its input and output layers.

Key Hyperparameters

- **Number of hidden units:** Defines the network's capacity and complexity.
- **L2 Regularization (Weight Decay):** Prevents overfitting by penalizing large weights.

Training via Backpropagation

The algorithm iteratively adjusts network weights by propagating the error gradient backwards from the output layer through hidden layers to minimize prediction errors.

Overfitting Mitigation

L2 regularization combats overfitting by adding a penalty proportional to the square of the magnitude of weights, discouraging overly complex models.

When to Use MLP

Ideal for non-linear classification problems and moderate-to-large datasets where complex relationships between variables need to be learned.

Further Reading

Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann. — Chapter 9: Classification: Advanced Methods (Section on Neural Networks)

MLP Neural Network: cla_mlp

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
# size: units in the hidden layer
# decay: L2 regularization
model <- cla_mlp("Species", slevels,
                 size=3, decay=0.03)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 0.983

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 0.967
```

Step by Step

- Use `cla_mlp()` with `size` (hidden units) and `decay` (L2 regularization)
- `size=3`: 3 neurons in the hidden layer
- `decay=0.03`: penalizes large weights to reduce overfitting
- Training accuracy: **0.983** | Test accuracy: **0.967**
- Feedforward network trained via backpropagation

📌 **Key Concept:** MLP is a **feedforward neural network**. The `decay` parameter applies L2 regularization to control overfitting by penalizing excessively large weights.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

